

Zusammenfassung

0.0.1. Beispiel: File Streaming Service

Client (Program.cs)

```

1  using AsyncClientStreamingCall<FileDto, Empty> call = client.SendFiles();
2
3  // The port number(5001) must match the port of the gRPC server.
4  GrpcChannel channel = GrpcChannel.ForAddress("https://localhost:5001");
5  StreamingService.StreamingClient client = new(channel); // Client that every function uses
6
7  // Run all streaming functions
8  await TestServerStreaming(client); await TestClientStreaming(client); await TestBiDirectionalStreaming(client);
9  WriteLine("Press any key to exit..."); ReadKey();
10
11 // Stream files from server
12 static async Task TestServerStreaming(StreamingService.StreamingClient client) {
13     WriteLine(nameof(TestServerStreaming));
14     using AsyncServerStreamingCall<FileDto> call = client.ReadFiles(new()); // Call object without Parameter
15     await foreach (FileDto message in call.ResponseStream.ReadAllAsync()) { // Read last written chunk
16         WriteLine($"File: {message.FileName}, Line Nr: {message.Line}, Content: {message.Content}");
17     }
18 }
19
20 // Stream files to server
21 static async Task TestClientStreaming(StreamingService.StreamingClient client) {
22     WriteLine(nameof(TestClientStreaming));
23     string[] files = Directory.GetFiles(@"Files"); // Get all files in folder
24     foreach (string file in files) { // Open every file
25         string content; int line = 0; using StreamReader reader = File.OpenText(file);
26         while ((content = await reader.ReadLineAsync()) != null) {
27             line++; // Read every line
28             FileDto reply = new() { FileName = file, Line = line, Content = content, }; // Write into Protobuf
29             await call.RequestStream.WriteAsync(reply); // Write protobuf to stream
30         }
31     }
32     // Closing the stream is required
33     await call.RequestStream.CompleteAsync(); // No more messages to come (server exits foreach-Loop)
34     Empty result = await call; // Wait until service method is terminated / Get the result
35 }
36
37 // Send and receive files at the same time
38 static async Task TestBiDirectionalStreaming(StreamingService.StreamingClient client) {
39     WriteLine(nameof(TestBiDirectionalStreaming));
40     using AsyncDuplexStreamingCall<FileDto, FileDto> call = client.RoundtripFiles();
41     // Read
42     Task readTask = Task.Run(async () => {
43         await foreach (FileDto message in call.ResponseStream.ReadAllAsync()) {
44             WriteLine($"File: {message.FileName}, Line Nr: {message.Line}, Content: {message.Content}");
45         }
46     });
47     // Write
48     string[] files = Directory.GetFiles(@"Files");
49     foreach (string file in files) {
50         string content; int line = 0; using StreamReader reader = File.OpenText(file);
51         while ((content = await reader.ReadLineAsync()) != null) {
52             line++;
53             FileDto reply = new() { FileName = file, Line = line, Content = content, };
54             await call.RequestStream.WriteAsync(reply);
55         }
56     }
57     // Required
58     await call.RequestStream.CompleteAsync(); // No more messages to come (server exits foreach-Loop)
59     await readTask; // Wait until service method is terminated / all messages are received by client
60 }

```

Server

```

1  public class StreamingService : FileStreamingService.FileStreamingServiceBase {
2      // Read files from disk and send to the client
3      public override async Task ReadFiles(
4          Empty request, // No parameters
5          IServerStreamWriter<FileDto> responseStream,
6          ServerCallContext context)
7      {
8          Empty request, // No parameters
9          IServerStreamWriter<FileDto> responseStream,
10         ServerCallContext context)
11     {
12         WriteLine(nameof(ReadFiles));
13         string[] files = Directory.GetFiles(@"..\11_FileStreamingFiles");
14
15         foreach (string file in files) {
16             string content; int line = 0;
17             using StreamReader reader = File.OpenText(file);
18
19             // Read until End of File
20             while ((content = await reader.ReadLineAsync()) != null) {
21                 line++;
22                 FileDto reply = new() { FileName = file, Line = line, Content = content, };
23                 await responseStream.WriteAsync(reply);
24             }
25         }
26     }
27
28     // Receive files that the client sends
29     public override async Task<Empty> SendFiles(
30         IAsyncStreamReader<FileDto> requestStream,
31         ServerCallContext context)
32     {
33         WriteLine(nameof(SendFiles));
34         await foreach (FileDto message in requestStream.ReadAllAsync()) { // Read last written chunk
35             WriteLine($"File: {message.FileName}, Line Nr: {message.Line}, Content: {message.Content}");
36         }
37         return new Empty(); // Empty result, nothing to return
38     }
39
40     // Send files back to the client
41     public override async Task RoundtripFiles(
42         IAsyncStreamReader<FileDto> requestStream,
43         IServerStreamWriter<FileDto> responseStream,
44         ServerCallContext context)
45     {
46         WriteLine(nameof(RoundtripFiles));
47         await foreach (FileDto message in requestStream.ReadAllAsync()) {
48             await responseStream.WriteAsync(message);
49             WriteLine($"File: {message.FileName}, Line Nr: {message.Line}, Content: {message.Content}");
50         }
51     }
52 }

```

Proto

```

1  syntax = "proto3";
2  import "google/protobuf/empty.proto";
3  option csharp_namespace = "FileStreaming";
4  package FileStreaming;
5
6  service StreamingService {
7      rpc ReadFiles (google.protobuf.Empty) returns (stream FileDto);
8      rpc SendFiles (stream FileDto) returns (google.protobuf.Empty);
9      rpc RoundtripFiles (stream FileDto) returns (stream FileDto);
10 }
11
12 message FileDto {
13     string file_name = 1;
14     int32 line = 2;
15     string content = 3;
16 }

```